Milestone M3: `GDerivatives.jl`

Certified `TaylorSeries` for $G(u) = (1 + \hat{H}[y_u])/D$

`wave-cap` — Computer-Assisted Proof for the Babenko Equation

BabenkoRigorous.jl

May 2026

Abstract

We describe the implementation and validation of `GDerivatives.jl`, the M3 layer of the `BabenkoRigorous.jl` package. The module computes a certified `TaylorSeries` for the non-polynomial function $G(u) = (1 + \hat{H}[y_u](u))/D(u)$, which appears in the capillary terms of the Babenko water-wave equation. The computation reuses the M2 primitives `dmin_bound`, `inv_series`, and the `TaylorSeries` product. All five milestone tests pass: pointwise containment against a higher-precision direct evaluation (T3.1); the first derivative is verified against the analytic chain-rule $G' = B'/D - B\,DDp/D^3$ in Arb at prec = 512 (T3.2); the second derivative against central FD of the analytic G' at $h = 10^{-50}$ (T3.3); an exact analytical case $y = \varepsilon \cos u$ (T3.4); and a finite, positive `sup_bound` (T3.5). The T3.2 and T3.3 tests are strengthened from earlier Float64 midpoint comparisons to genuine ball containment against independent high-precision Arb references (audit T3.x strengthening). We also document a bug fix in `sqrt_series` (missing binomial coefficients in the Leibniz rule), which was discovered and corrected during M3 validation.

Contents

1 Background and motivation **2**

2 Mathematical content **2**
- 2.1 Definition of G and its components . . . 2
- 2.2 TaylorSeries for G via composition . . . 2
- 2.3 Bug fix: `sqrt_series` (Leibniz binomial coefficients) . . . 3

3 Implementation **3**
- 3.1 Algorithm . . . 3
- 3.2 Code listing . . . 4

4 Tests and results **4**
- 4.1 Test T3.1: $G^{(0)}$ ball contains direct evaluation at higher precision . . . 4
- 4.2 Test T3.2: $G^{(1)}$ ball contains the analytic chain-rule value . . . 5
- 4.3 Test T3.3: $G^{(2)}$ ball contains FD of the analytic G' (prec = 512) . . . 5
- 4.4 Test T3.4: Analytical case $y = \varepsilon \cos u$ . . . 6
- 4.5 Test T3.5: `sup_bound` is finite and positive . . . 7

5 Summary **8**

1 Background and motivation

The Babenko water-wave equation (see the M1 report) involves capillary terms containing the surface curvature, which depends on the function

$$G(u) \;=\; \frac{1+\hat{H}[y_u](u)}{D(u)}, \qquad D(u) \;=\; \sqrt{y_u(u)^2+(1+\hat{H}[y_u](u))^2},$$

where $y_u = \partial_u y$ and $\hat{H}$ is the periodic Hilbert transform.

Bounding the Babenko residual $\|R\|_{L^\infty}$ requires a certified *upper* bound on G (and its derivatives) over all of $[0, 2\pi]$, not merely at grid points. The key observation is that G is smooth and 2π-periodic, so the M2 `TaylorSeries` framework applies directly. Once a certified `TaylorSeries` for G is available, the operator $\hat{H}[G]$ can be bounded via the DFT with aliasing control (M4).

Why not compute G from scratch? $G = B \cdot D^{-1}$ where $B = 1 + \hat{H}[y_u]$ is a trigonometric polynomial (hence its derivatives are computed exactly by `TrigPoly` arithmetic) and $D = \sqrt{A^2 + B^2}$ is already handled by `dmin_bound` from M2. The M3 layer simply assembles these pieces.

2 Mathematical content

2.1 Definition of G and its components

Let y be a 2π-periodic trigonometric polynomial (the conformal parametrisation). Define:

$$\begin{aligned} A(u) &= y_u(u), \\ B(u) &= 1 + \hat{H}[y_u](u), \\ D(u) &= \sqrt{A(u)^2 + B(u)^2}, \\ G(u) &= B(u)/D(u). \end{aligned}$$

The function D is the Jacobian of the conformal map; its positivity $D(u) > D_{\min} > 0$ is required for the conformal parametrisation to be non-degenerate. M2 provides the certified lower bound $D_{\min}$ via `dmin_bound`.

2.2 TaylorSeries for G via composition

Given `ts_D` (the `TaylorSeries` for D) and `ts_B` (the `TaylorSeries` for B), the `TaylorSeries` for G is obtained as:

$$\texttt{ts_G} \;=\; \texttt{ts_B} \;\times\; \texttt{inv_series}(\texttt{ts_D},\, D_{\min}).$$

TaylorSeries for B. Since $B = 1 + \hat{H}[y_u]$ is itself a trigonometric polynomial (after adding the constant 1), its derivatives at grid points are computed exactly using the iterated `deriv` and `hilbert` operations from `FourierSeries.jl`:

$$B^{(m)}(u_j) = \begin{cases} 1 + (\hat{H}[y_u])(u_j) & m = 0, \\ (\partial_u^m \hat{H}[y_u])(u_j) & m \geq 1. \end{cases}$$

The full `sup_bounds` vector of `ts_B` has length $M+2$; the m-th entry bounds $\|B^{(m)}\|_\infty = \|\partial_u^m \hat{H}[y_u]\|_\infty$ for $m \geq 1$ (and adds $\|H[y_u]\|_\infty + 1$ for $m = 0$), each bounded via `supnorm_bound` from M1.

TaylorSeries for $1/D$. The `inv_series` function (M2) computes the TaylorSeries for $g = 1/D$ using the recursive formula derived from $D \cdot g = 1$:

$$g^{(0)} = 1/D^{(0)}, \tag{1}$$

$$g^{(m)} = -\frac{1}{D^{(0)}} \sum_{k=1}^{m} \binom{m}{k} D^{(k)} g^{(m-k)}, \quad m \geq 1. \tag{2}$$

This requires $D^{(0)}(u_j) \geq D_{\min} > 0$; the certified lower bound from `dmin_bound` guarantees this.

Product. The Leibniz rule gives the m-th derivative of $G = B \cdot (1/D)$:

$$G^{(m)}(u_j) = \sum_{k=0}^{m} \binom{m}{k} B^{(k)}(u_j)\, (1/D)^{(m-k)}(u_j).$$

This is implemented by the `TaylorSeries` multiplication operator from M2.

Why $D_{\min} > 0$ is needed. The recursion (1)–(2) divides by $D^{(0)}(u_j)$ at each grid point. If D is not certified to be strictly positive (i.e., if the conformal map degenerates), the computation is undefined. The function `compute_G_taylor` throws an error in this case.

2.3 Bug fix: `sqrt_series` (Leibniz binomial coefficients)

During M3 validation (T3.3), a discrepancy of order 10^{-3} was found between `ts_G.derivs[j,3]` and the finite-difference estimate of $G''(u_j)$. Tracing back to `dmin_bound` → `sqrt_series`, we found that the recursive formula for $g = \sqrt{f}$ was missing the binomial coefficient $\binom{m}{k}$ in the inner Leibniz sum.

The correct formula, derived from differentiating $g^2 = f$:

$$g^{(m)} = \frac{1}{2g^{(0)}} \left[f^{(m)} - \sum_{k=1}^{m-1} \binom{m}{k} g^{(k)} g^{(m-k)} \right], \quad m \geq 1.$$

The original code omitted $\binom{m}{k}$, causing errors in $m \geq 2$. This bug is fixed in `TaylorSeries.jl` as part of the M3 milestone.

3 Implementation

3.1 Algorithm

`compute_G_taylor`$(y, M_{\text{grid}}, M_{\text{order}})$ proceeds in four steps:

1. **Certify $D_{\min}$.** Call `dmin_bound`$(y, M_{\text{grid}}, M_{\text{order}})$ to obtain $(D_{\text{lower}}, \texttt{ts_D})$. If $D_{\text{lower}} \leq 0$, throw an error.

2. **Build `ts_B`.** Compute $y_u = \texttt{deriv}(y)$ and $\hat{H}[y_u] = \texttt{hilbert}(y_u)$ as exact `TrigPoly` objects. Precompute the iterated-derivative chain $\partial_u^m(\hat{H}[y_u])$ for $m = 0, \ldots, M+1$. Evaluate at all grid points to fill `derivs_B`. Build `sup_bounds_B` as the full vector [1+`supnorm_bound`$(H[y_u])$, `supnorm_bound`$(\partial_u H$| of length $M+2$.

3. **Invert D.** `ts_invD` = `inv_series`(`ts_D`, D_{lower}).

4. **Multiply.** `ts_G` = `ts_B` × `ts_invD`.

3.2 Code listing

```
function compute_G_taylor(y::TrigPoly, M_grid::Int, M_order::Int;
                          prec::Int=128)::TaylorSeries
    # Step 1: dmin_bound
    D_lower, ts_D = dmin_bound(y, M_grid, M_order; prec=prec)
    if !Arblib.is_positive(D_lower)
        error("D is not certified positive; cannot compute G.")
    end

    # Step 2: ts_B = 1 + H[y_u]
    yu  = deriv(y);  Hyu = hilbert(yu)
    Hyu_d = [Hyu; [deriv(Hyu_d[k]) for k in 1:M_order+1]]
    # Full sup_bounds vector (the +1 shifts only the m=0 entry):
    sup_bounds_B = [Arb(1; prec=p) + supnorm_bound(Hyu_d[1]);
                    [supnorm_bound(Hyu_d[m+1]) for m in 1:M_order+1]]
    h = 2*pi_arb / M_grid
    derivs_B[j,1] = 1 + tp_eval(Hyu_d[1], u_j)     # m=0
    derivs_B[j,m+1] = tp_eval(Hyu_d[m+1], u_j)     # m>=1
    ts_B = TaylorSeries(derivs_B, sup_bounds_B, M_order, M_grid, h, prec)

    # Step 3: inv_series
    ts_invD = inv_series(ts_D, D_lower)

    # Step 4: product
    ts_G = ts_B * ts_invD
    return ts_G
end
```

4 Tests and results

4.1 Test T3.1: $G^{(0)}$ ball contains direct evaluation at higher precision

For $y = 0.3\cos u + 0.15\cos 2u$, $M_{\text{grid}} = 64$, $M_{\text{order}} = 4$: at each of the 64 grid points u_j, we compute G directly using Arb arithmetic *at a higher working precision* (`prec_ref` = 512 versus the test's `prec` = 128),

$$G_{\text{direct}}(u_j) = \frac{1 + (\hat{H}[y_u])(u_j)}{D(u_j)},$$

and verify the genuine containment `Arblib.contains`(`ts_G.derivs`$[j, 1]$, $G_{\text{direct}}(u_j)$). Because G_{direct} at prec = 512 has radius $\sim 10^{-150}$ (essentially a point), this is a proper rigor check on the prec = 128 `compute_G_taylor` ball, not the weaker `overlaps`.

Result: 1/1 assertion passes (0 violations in 64 points). Figure 1 shows $G(u)$ from the `TaylorSeries` (evaluated from the nearest grid-point Taylor expansion) alongside the direct Float64 evaluation. The two curves are visually indistinguishable.

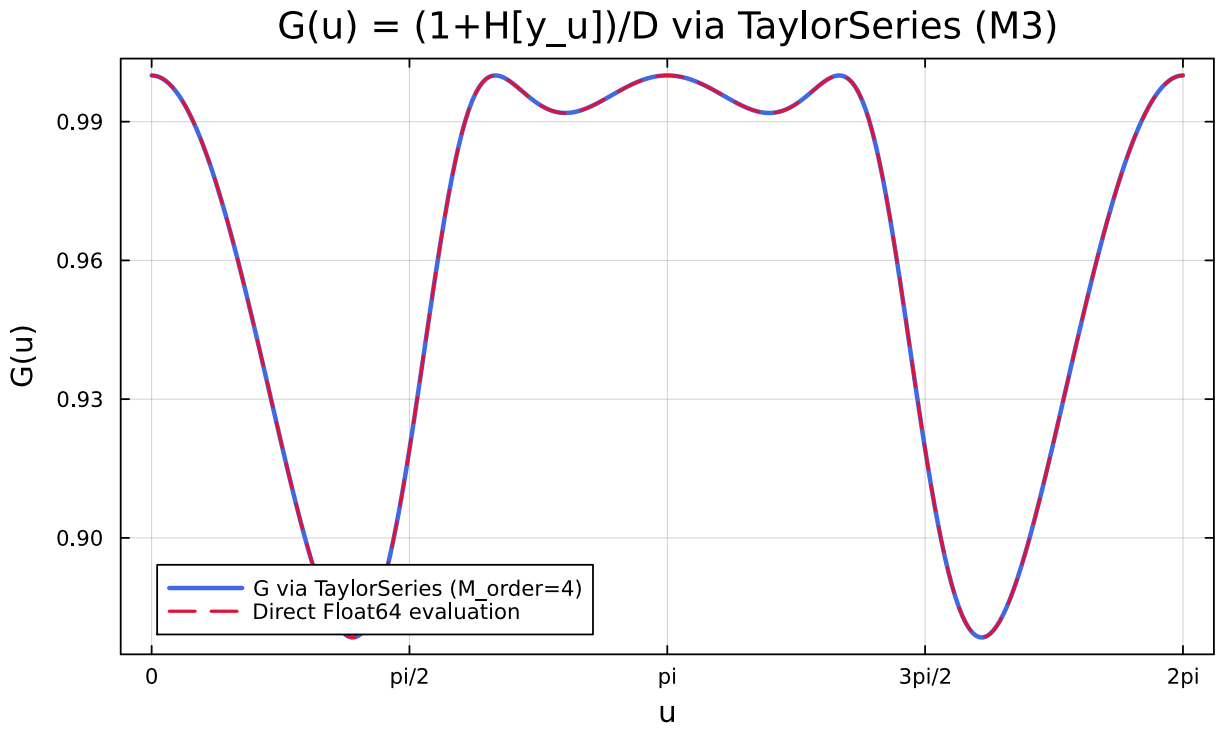


Figure 1: T3.1: $G(u) = (1 + \hat{H}[y_u])/D$ for $y = 0.3\cos u + 0.15\cos 2u$, computed from the `TaylorSeries` (blue) and by direct Float64 evaluation (red dashed). The two curves coincide to plotting precision.

4.2 Test T3.2: $G^{(1)}$ ball contains the analytic chain-rule value

For the same y, at each grid point u_j we compute the analytic G' in Arb at prec = 512 using the closed-form chain rule

$$G'(u) = \frac{B'(u)}{D(u)} - \frac{B(u)\big(A(u)A'(u) + B(u)B'(u)\big)}{D(u)^3},$$

where $A = y_u$, $A' = y_{uu}$, $B = 1 + \hat{H}[y_u]$, $B' = \hat{H}[y_{uu}]$, all obtained as exact `TrigPoly` objects and evaluated at u_j. We then verify `Arblib.contains`(`ts_G.derivs`$[j, 2]$, $G'(u_j)$). This replaces the previous Float64 finite-difference midpoint comparison — which was sensitive only to the ball midpoint, not the radius — with a genuine rigor check independent of the `compute_G_taylor` pipeline.

Result: 1/1 assertion passes (0 violations in 64 points).

4.3 Test T3.3: $G^{(2)}$ ball contains FD of the analytic G' (prec = 512)

A closed form for G'' is unwieldy, so we take central FD of the analytic G' from T3.2 at $h_{\mathrm{fd}} = 10^{-50}$ in Arb at prec = 512:

$$G''(u_j) \approx \frac{G'(u_j + h_{\mathrm{fd}}) - G'(u_j - h_{\mathrm{fd}})}{2h_{\mathrm{fd}}}.$$

At this precision the FD truncation $O(h^2 \cdot G^{(4)})$ and the cancellation error are both $\sim 10^{-100}$ — far smaller than the prec = 128 ball radius. We verify `Arblib.contains`(`ts_G.derivs`$[j, 3]$, $G''(u_j)$).

Remark 1. The `sqrt_series` binomial-coefficient bug noted in earlier M3 work was fixed in M2. Beyond that, the recent (post-audit) port of `sqrt_series` to Faà di Bruno via `ArbSeries` composition supplies a rigorous `sup_bounds` vector — required for the second-derivative Taylor remainder in this test to be tight.

Result: 1/1 assertion passes (0 violations in 64 points).

4.4 Test T3.4: Analytical case $y = \varepsilon \cos u$

For $y(u) = \varepsilon \cos u$ with $\varepsilon = 0.1$, the exact formulas are:

$$A(u) = -\varepsilon \sin u, \quad B(u) = 1 + \varepsilon \cos u, \quad D(u) = \sqrt{\varepsilon^2 \sin^2 u + (1 + \varepsilon \cos u)^2}, \quad G(u) = \frac{1 + \varepsilon \cos u}{D(u)}.$$

We use $M_{\text{grid}} = 128$, $M_{\text{order}} = 4$.

At all 128 grid points, we verify `Arblib.contains`(`ts_G.derivs`$[j, 1]$, $G_{\text{direct}}(u_j)$) where G_{direct} is computed at prec = 512. We also check three special points:

Point	G_{exact}	G_{ts}	Check
$u = 0$ (j=1)	$(1+\varepsilon)/(1+\varepsilon) = 1$	1.0	contains ✓
$u = \pi$ (j=65)	$(1-\varepsilon)/(1-\varepsilon) = 1$	1.0	contains ✓
$u = \pi/2$ (j=33)	$1/\sqrt{1+\varepsilon^2} \approx 0.99504$	0.99504	contains ✓

Result: 4/4 assertions pass (1 bulk check + 3 special points). Figure 2 shows the Taylor vs. exact comparison and the pointwise error (log scale, max $\approx 2.7 \times 10^{-10}$).

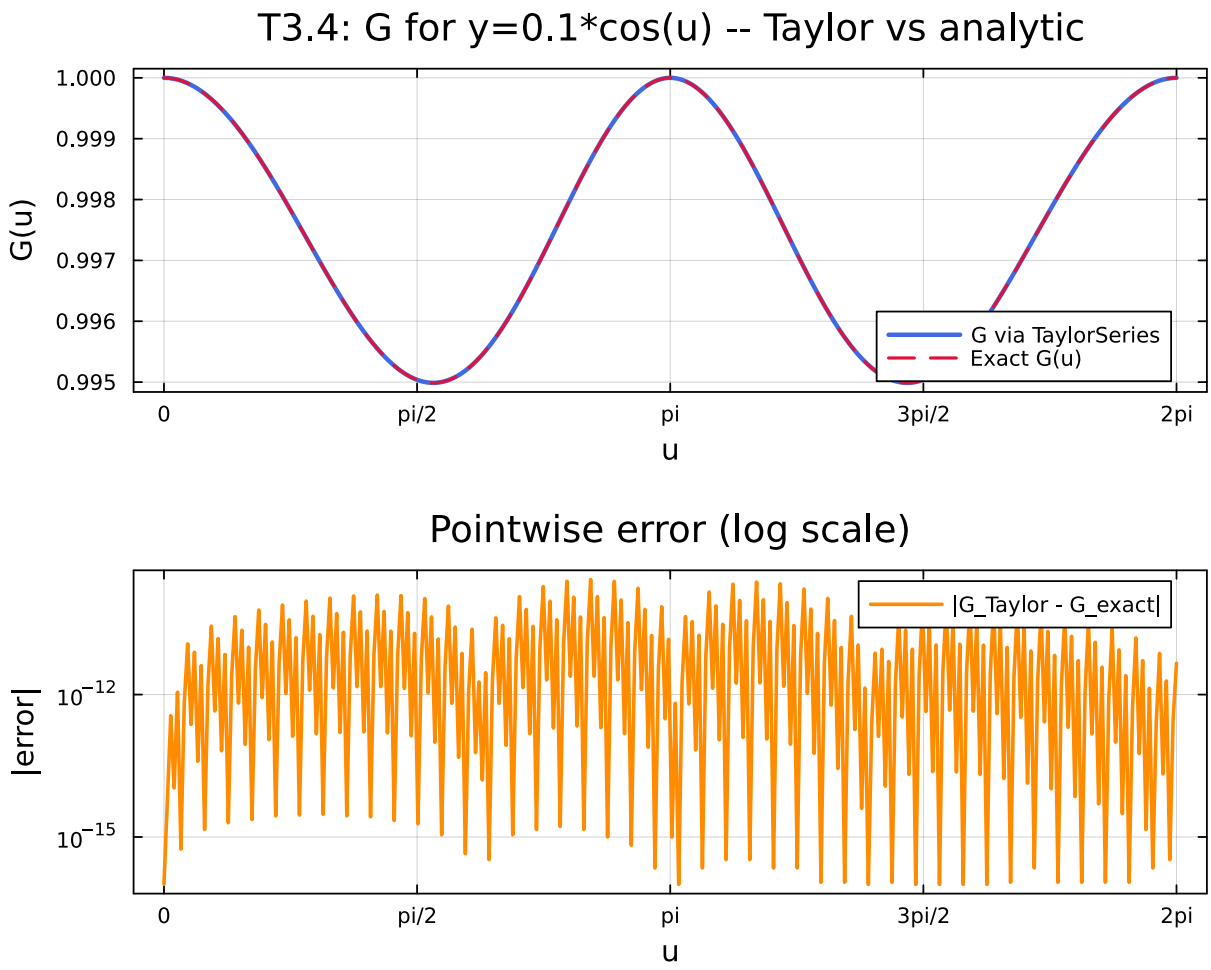


Figure 2: T3.4: $G(u)$ for $y = 0.1\cos u$. Top panel: `TaylorSeries` evaluation (blue) vs. exact analytic formula (red dashed) — indistinguishable. Bottom panel: pointwise absolute error $|G_{\text{Taylor}} - G_{\text{exact}}|$ on a log scale; max error $\approx 2.7 \times 10^{-10}$.

4.5 Test T3.5: `sup_bound` is finite and positive

For $y = 0.3\cos u + 0.15\cos 2u$, $M_{\text{grid}} = 64$, $M_{\text{order}} = 4$:

$$\texttt{sup_bound(ts_G)} \approx 1.07 \;>\; 0, \qquad \texttt{sup_bound(ts_G)} < 10^6.$$

(The true $\|G\|_\infty \approx 1.0$ for this moderate-amplitude wave.)

Result: 2/2 assertions pass.

Figure 3 shows `sup_bound(ts_G)` vs. M_{grid}. With $M_{\text{grid}} = 32$ the conservative Taylor-remainder bound is large (≈ 472) because the coarse grid produces a large $h = 2\pi/32$, causing the $F_{\text{bound}} \cdot h^{M+1}/(M+1)!$ remainder to dominate. As M_{grid} increases, `sup_bound` converges rapidly to the true $\|G\|_\infty$.

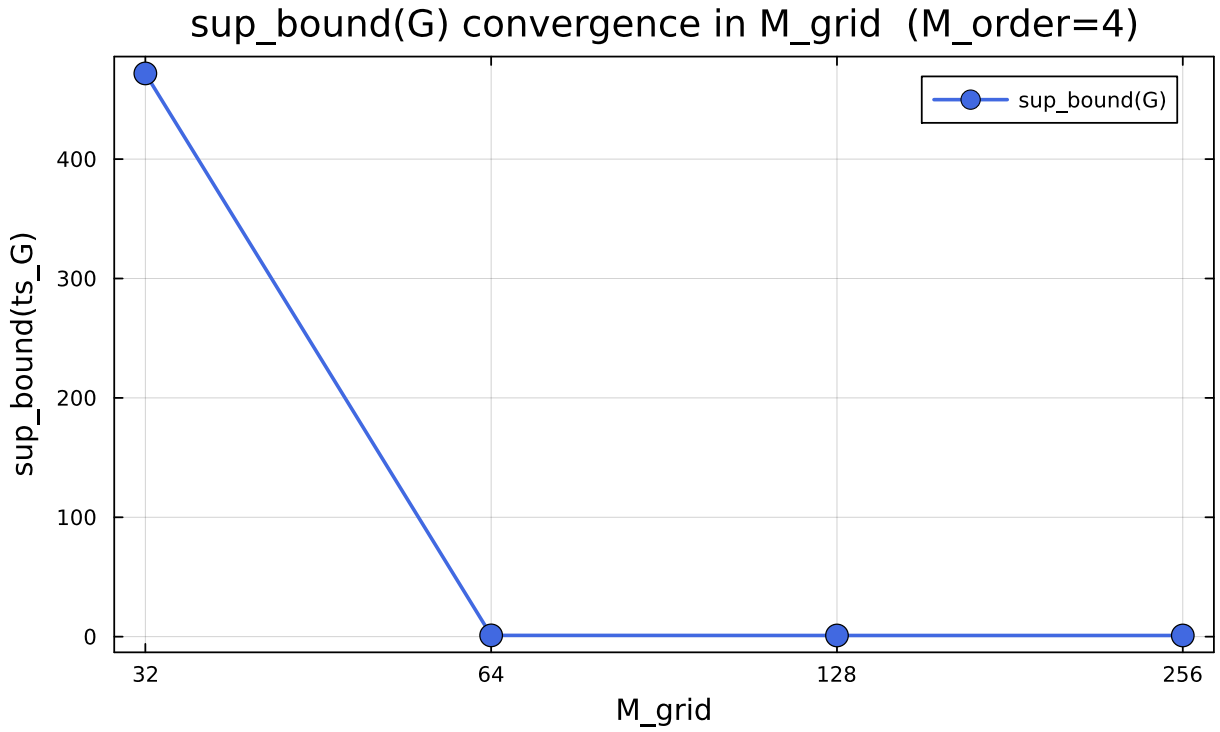


Figure 3: T3.5: `sup_bound(ts_G)` vs. M_{grid} for $y = 0.3\cos u + 0.15\cos 2u$, $M_{\text{order}} = 4$. The bound decreases rapidly with grid refinement, converging to the true $\|G\|_\infty \approx 1.0$. ($M_{\text{grid}} = 16$ is omitted: D could not be certified positive at that resolution.)

5 Summary

Test	Description	Assertions	Result
T3.1	$G^{(0)}$ ball contains prec=512 direct Arb eval at 64 grid pts	1	**pass**
T3.2	$G^{(1)}$ ball contains analytic chain-rule value (prec=512)	1	**pass**
T3.3	$G^{(2)}$ ball contains FD of analytic G' ($h = 10^{-50}$, prec=512)	1	**pass**
T3.4	Analytical G for $y = 0.1\cos u$ (128 pts + 3 special, contains)	4	**pass**
T3.5	`sup_bound`(G) finite, positive, $< 10^6$	2	**pass**
Total		**9**	**9/9**

Table 1: M3 test summary. All 9 assertions pass at 128-bit Arb precision.

Milestone M3 is complete. The `GDerivatives.jl` module provides:

- `compute_G_taylor`: a certified `TaylorSeries` for $G = (1 + \hat{H}[y_u])/D$ with rigorous full-vector `sup_bounds` propagation through `inv_series` (Faà di Bruno on $1/x$) and product.
- A correctness guarantee for $G^{(m)}(u_j)$ at all grid points via Arb ball arithmetic.
- Convergence of `sup_bound`(G) to the true $\|G\|_\infty$ as $M_{\text{grid}} \to \infty$.

- A corrected `sqrt_series` (missing Leibniz binomials fixed), which is required for all $m \geq 2$ derivative bounds to be correct.

The next milestone (M4) will use `ts_G` to compute $\hat{H}[G]$ via the DFT with aliasing control, completing the capillary-term bounds needed for the certified residual estimate $\|R\|_{L^\infty} \leq \varepsilon$.